

Computer Science

01. OS & CA

1주차: OS 개요 & 컴퓨터구조 기초

오늘의 핵심 질문

? 핵심 질문

"우리가 작성한 `hello.py` 파일은 저장장치에 잠들어 있다가 어떻게 실행되어 화면에 글자를 띄울까?"

이 질문 하나를 풀기 위해 CPU, RAM, SSD, 운영체제(OS), 시스템 콜(System Call)의 개념을 연결해 이해합니다.

🎯 학습 목표

- ✓ 컴퓨터 5대 핵심 구성 요소의 역할을 생활 속 비유와 함께 파악하기
- ✓ RAM(주기억장치)과 SSD(보조기억장치)의 차이와 프로그램 적재 흐름 이해하기
- ✓ 운영체제(OS)가 하드웨어를 관리하고 보호하는 원리 알기
- ✓ Python 코드가 실행될 때 운영체제와 하드웨어가 협력하는 전체 흐름 그리기

목차 INDEX

01 CPU

- 2진수와 16진수
- 논리게이트 기초
- ALU
- 캐시 & 레지스터
- CPU 구조

02 컴퓨터구조론

- RAM과 SSD의 차이
- 메모리 주소
- 메모리 계층 구조
- 폰 노이만 구조와 하버드 구조

03 운영체제

- 시스템 계층 구조
- 커널
- 셸
- 사용자 모드와 커널 모드
- 시스템 콜

04 코드 실행 과정

- 컴파일러와 인터프리터
- Python이 내부에서 실행되는 과정
- 프로그래밍 언어의 급

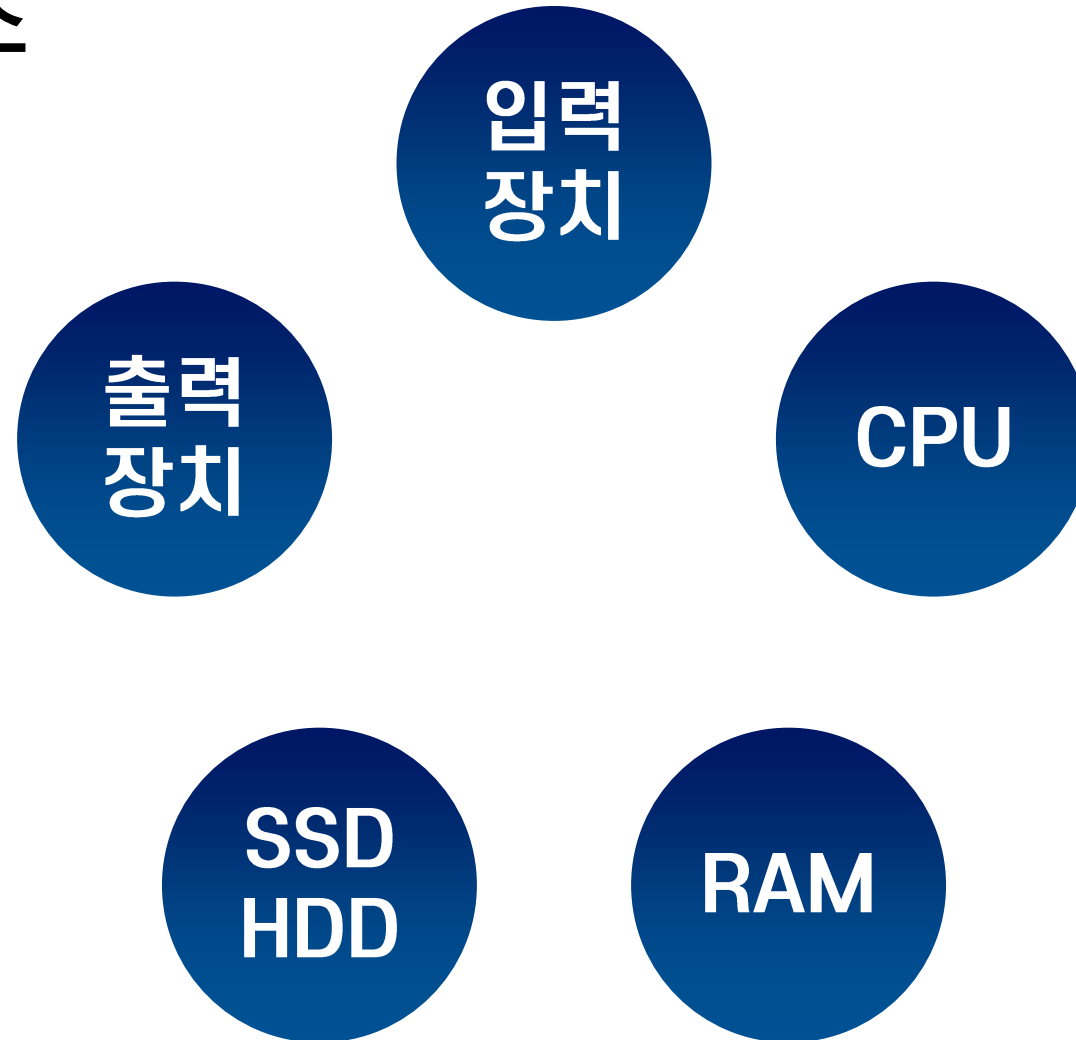
The background is a solid dark blue with abstract geometric shapes. A large, light blue curved shape is on the right side, and a dark blue diagonal band runs from the bottom left towards the top right.

01

CPU

Central Processing Unit

컴퓨터의 5대 요소



각 부품의 역할

CPU

계산하고 명령하는 숙련된 작업자

실제로 생각하고 계산하는 두뇌입니다.
엄청나게 빠른 속도로 연산을 수행하지만,
책상이 없으면 일할 수 없습니다.

RAM

현재 일하는 넓은 작업용 책상

지금 당장 실행 중인 프로그램과 서류를
올려두는 책상입니다. 책상이 넓을수록
많은 프로그램을 동시에 띄울 수 있습니다.

SSD

자료를 보관하는 거대한 서랍장

전원이 꺼져도 자료가 지워지지 않는 창고입니다.
평소 쓰지 않는 파이썬 파일, 게임, 사진 등이
안전하게 보관됩니다.

각 부품의 유기적 연결



CPU는 SSD에 있는 파일을 직접 읽지 못합니다! 반드시 **RAM**을 거쳐야만 코드를 실행할 수 있습니다.

2진수와 16진수

2진수 (Binary)

컴퓨터 내부의 모든 데이터는 전기가 흐름(1)과 흐르지 않음(0)으로 표현됩니다.

```
0b1101 # 앞에 '0b'가 붙으면 2진수
```

하지만 2진수는 자릿수가 너무 길어져서 사람이 읽고 쓰기 불편합니다.

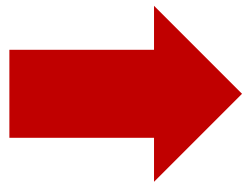
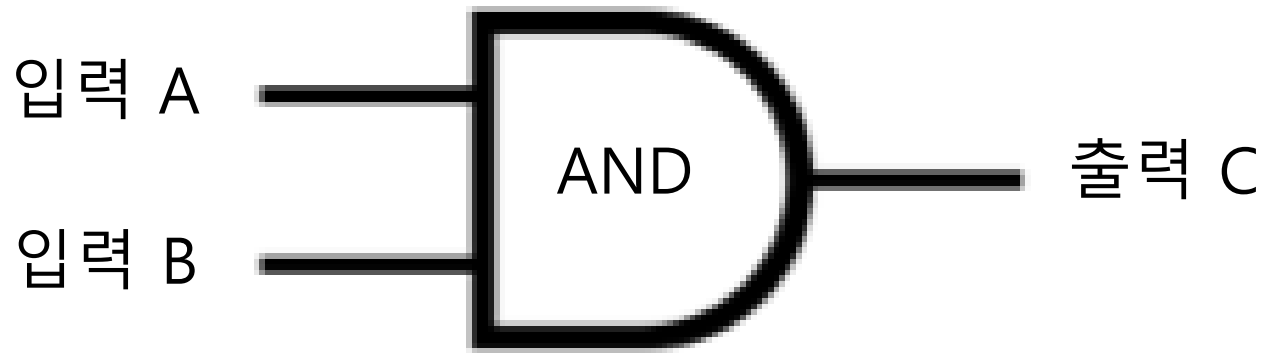
16진수 (Hexadecimal)

긴 2진수를 사람이 읽기 쉽게 묶어서 표현하는 방식입니다.

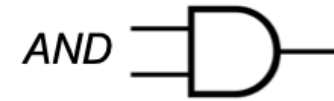
```
0x255A4F # 앞에 '0x'가 붙으면 16진수
```

메모리 주소, 색상 코드(#FF5733), 디버깅 화면에서 아주 자주 등장합니다.

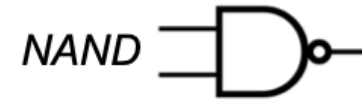
논리게이트 기초



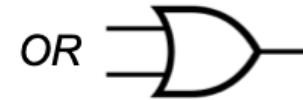
입력 A와 입력 B가 둘다 True 이면 출력 C는 True
(if input A **AND** input B are True, then output C is True)



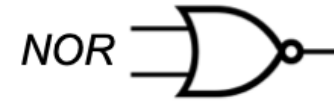
X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



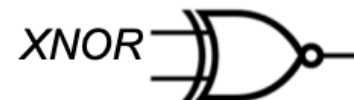
X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



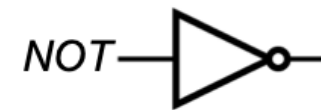
X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



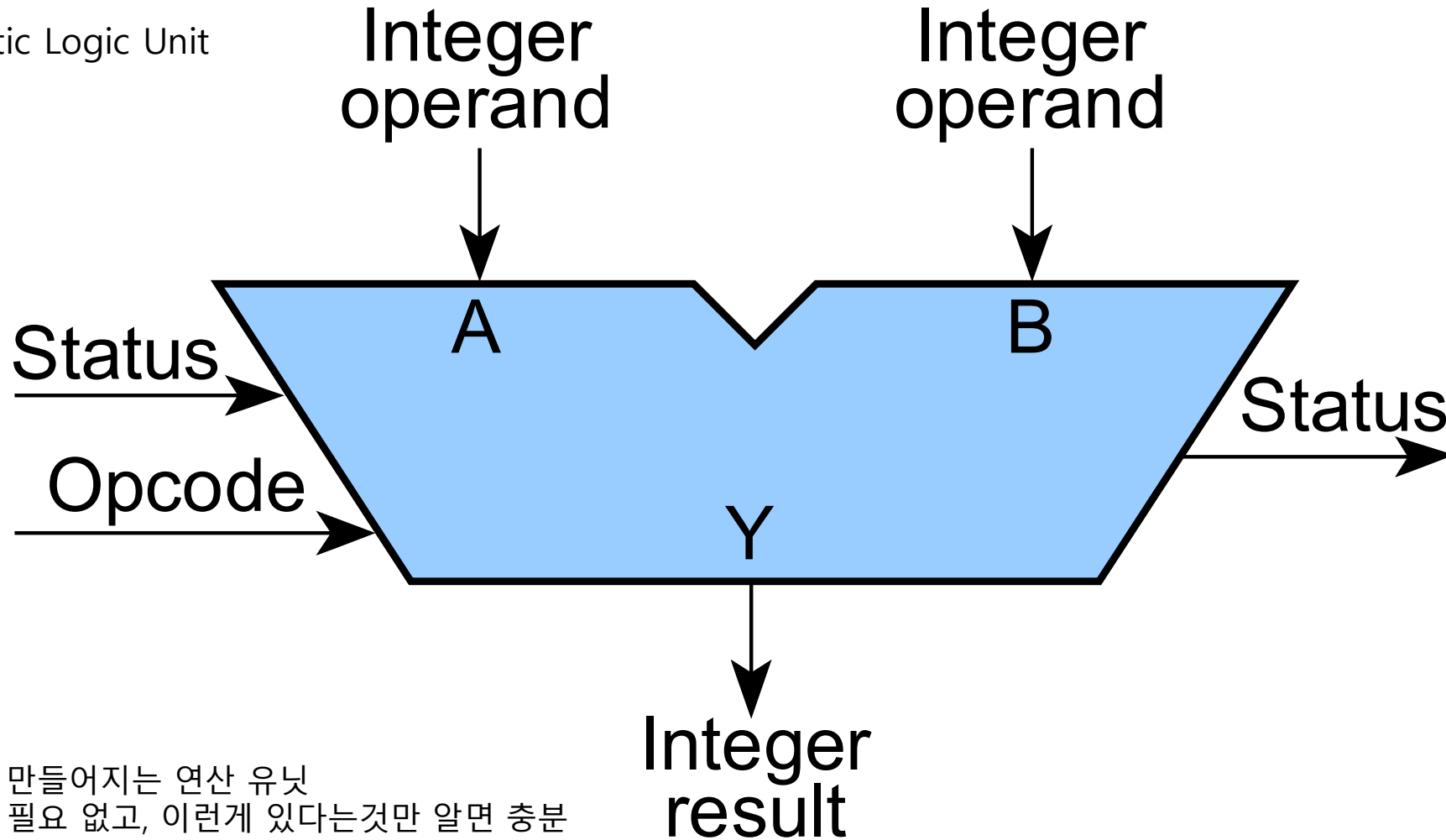
X	Y	OUT
0	0	0
0	1	0
1	0	0
1	1	1



X	OUT
0	1
1	0

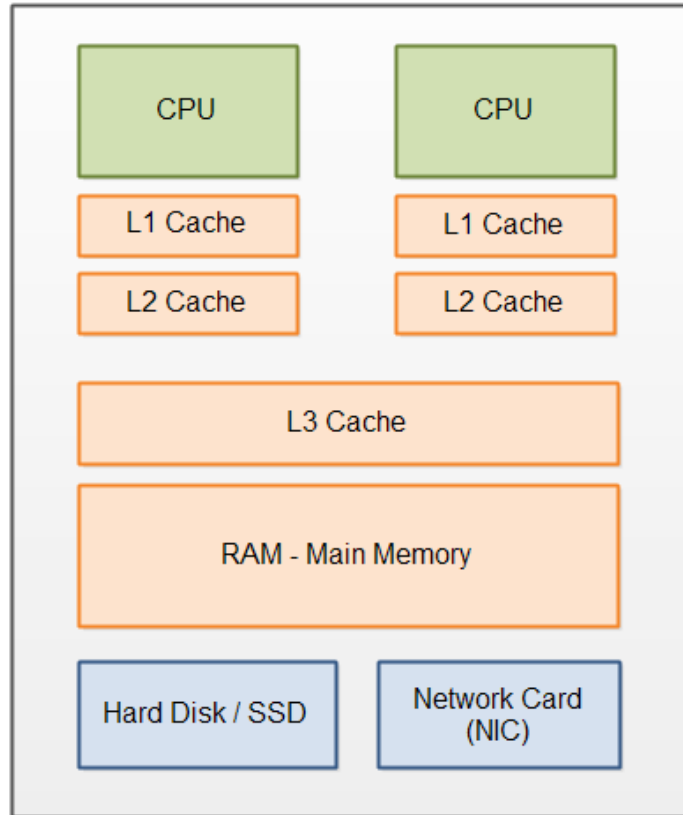
01 CPU Central Processing Unit

ALU Arithmetic Logic Unit

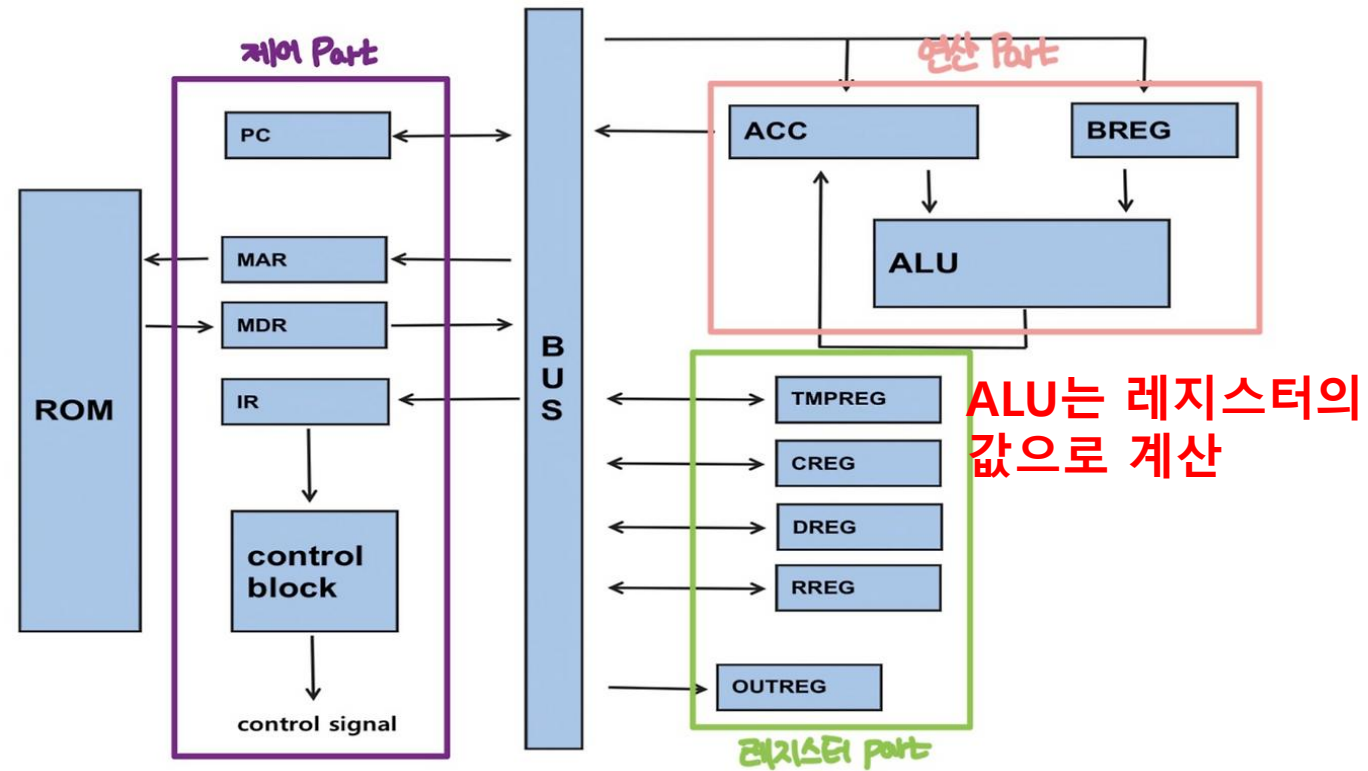


- 논리 게이트가 모여 만들어지는 연산 유닛
- 세부적인 내용은 알 필요 없고, 이렇게 있다는것만 알면 충분

Cache & Register



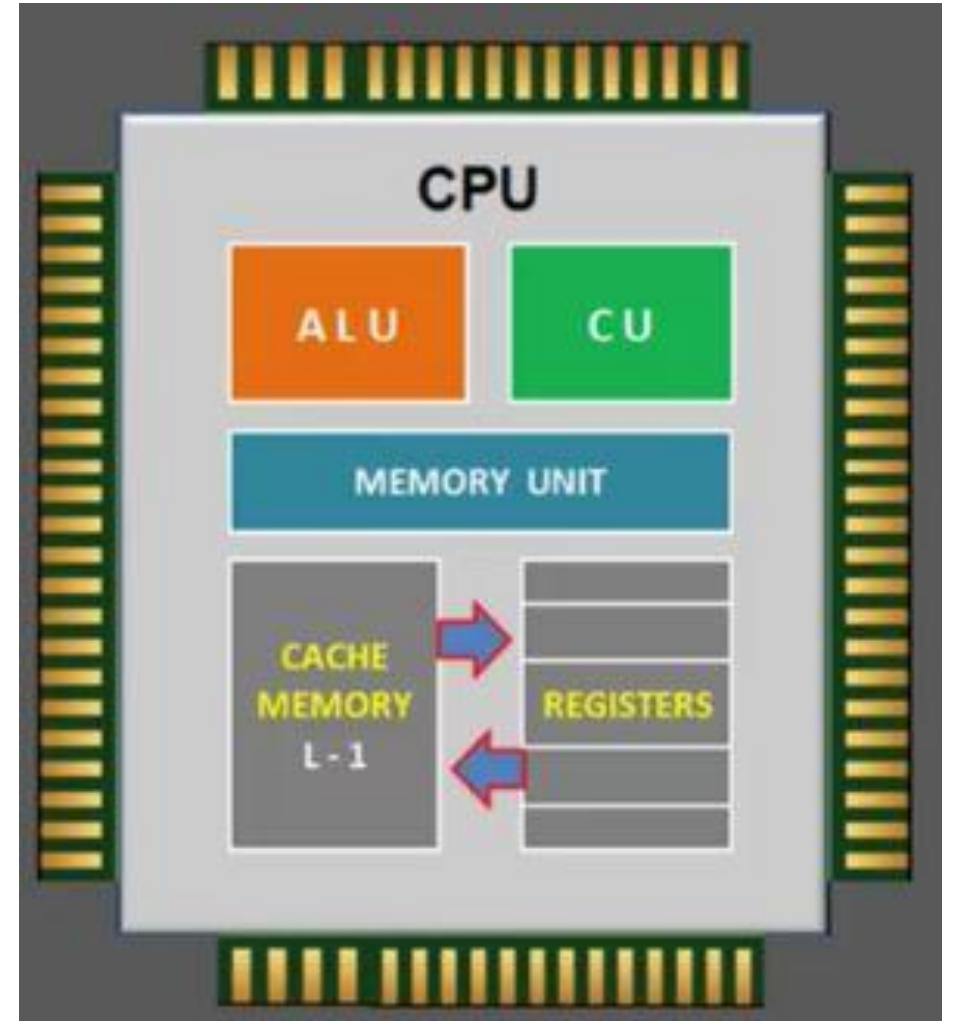
- CPU 칩 안에 들어있는 고속 임시 저장소
- RAM에서 가져온 데이터 임시 저장
- 빠른 CPU의 속도를 맞추기 위해 필요



- CPU 칩 안에 들어있는 초고속 임시 저장소
- 연산에 필요한 데이터와 주소 저장
- CPU가 명령 실행 및 연산 수행을 위해 사용

CPU의 구조

- 레지스터에 있는 값으로 ALU가 계산
- RAM에서 가져온 데이터를 캐시에 저장, 캐시된 데이터 중 필요한 데이터를 레지스터에 저장
- 모든 데이터 이동은 버스를 통해 이루어짐
- CU(Control Unit)은 이 모든 과정을 제어함



02

컴퓨터구조론

Computer Architecture

RAM과 SSD의 차이

구분	RAM (주기억장치)	SSD (보조기억장치)
핵심 역할	현재 실행 중인 프로그램과 데이터 임시 저장	프로그램, 파일, OS를 영구적으로 장기 저장
휘발성 (Volatile)	휘발성 (전원이 꺼지면 내용이 모두 사라짐)	비휘발성 (전원이 꺼져도 데이터가 안전하게 유지됨)
속도와 가격	매우 빠름, 용량당 가격이 비쌈	RAM에 비해 느림, 용량당 가격이 저렴함
파이썬 실행 시	파이썬 인터프리터와 코드가 적재 되어 실행됨	작성한 hello.py 소스 파일이 보관 됨

컴퓨터 전원을 켜면 SSD에 있던 운영체제 핵심 파일이 RAM으로 복사되면서 본격적인 부팅과 실행이 시작됨.

메모리 주소 Memory Address

메모리 주소란?

RAM은 수많은 방(저장 칸)들로 이루어진 거대한 아파트 단지과 같음

- RAM의 각 저장 위치에는 고유한 **주소(Address)**가 부여 됨
- CPU는 데이터를 찾거나 저장할 때 이 주소를 지목하여 찾아가
- 컴퓨터에서는 주소를 표현할 때 주로 **16진수**를 사용함
(예: 0x1000)

RAM 메모리 방 구조 예시

```
0x1000 : [ 변수 name의 데이터 'Alice' ]  
0x1004 : [ 명령어: 숫자 10을 더하라 ]  
0x1008 : [ 파이썬 인터프리터 코드 ]  
0x100C : [ 사용자 입력 버퍼 공간 ]
```

* 0x로 시작하는 표기는 16진수(Hexadecimal)를 의미함
CPU는 이 주소 번지수를 정확히 알고 접근함

메모리 계층 구조 Memory Hierarchy



계층 구조의 핵심 특징

위로 갈수록: 속도 ↑ 가격 ↑ 용량 ↓

아래로 갈수록: 속도 ↓ 가격 ↓ 용량 ↑

CPU는 연산 속도와 저장 공간의 한계를 극복하기 위해 계층 구조 활용

폰 노이만 아키텍처 VS 하버드 아키텍처 (참고)

🏠 폰 노이만 구조 (Von Neumann)

명령어와 데이터를 같은 메모리에 저장

현대 범용 컴퓨터(PC, 스마트폰)의 기본 설계 개념

프로그램 코드가 데이터와 함께 RAM에 나란히 올라감

CPU가 메모리에서 명령어와 데이터를
번갈아가며 가져와 실행

🔗 하버드 구조와의 비교 (참고)

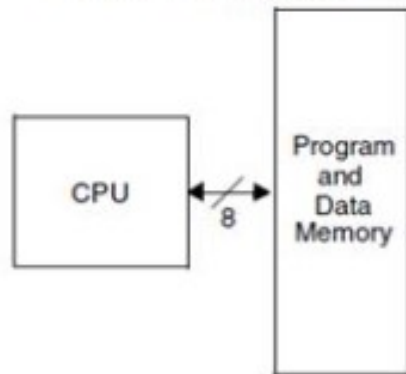
명령어 메모리와 데이터 메모리의 분리

명령어 전용 공간과 데이터 전용 공간이 따로 나뉨

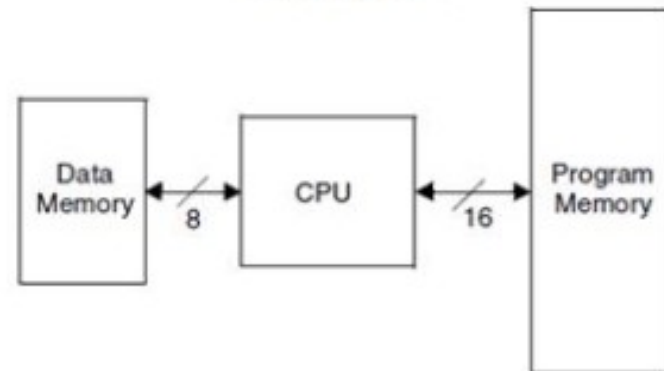
주로 특수 목적 장치(임베디드, 신호 처리기)에 쓰임

우리가 쓰는 일반 컴퓨터는 폰 노이만 구조 채택

von Neumann



Harvard



03

운영체제

Operating System

OS?

⚠ os가 없다면 어떻게 될까?

만약 운영체제가 없다면 파이썬 프로그램을 실행할 때마다 이러한 작업을 거쳐야 함

키보드 입력 신호를 직접 전압으로 읽어야 함

모니터 화면 픽셀 하나하나에 색상 값을 직접 써주어야 함

다른 프로그램과 메모리 공간이 겹치지 않게
직접 구역을 나눠야 함

🔧 운영체제(os)의 역할

운영체제(Windows, macOS, Linux 등)는 하드웨어를 숨기고 프로그래머와 사용자에게 편리한 인터페이스를 제공합니다.

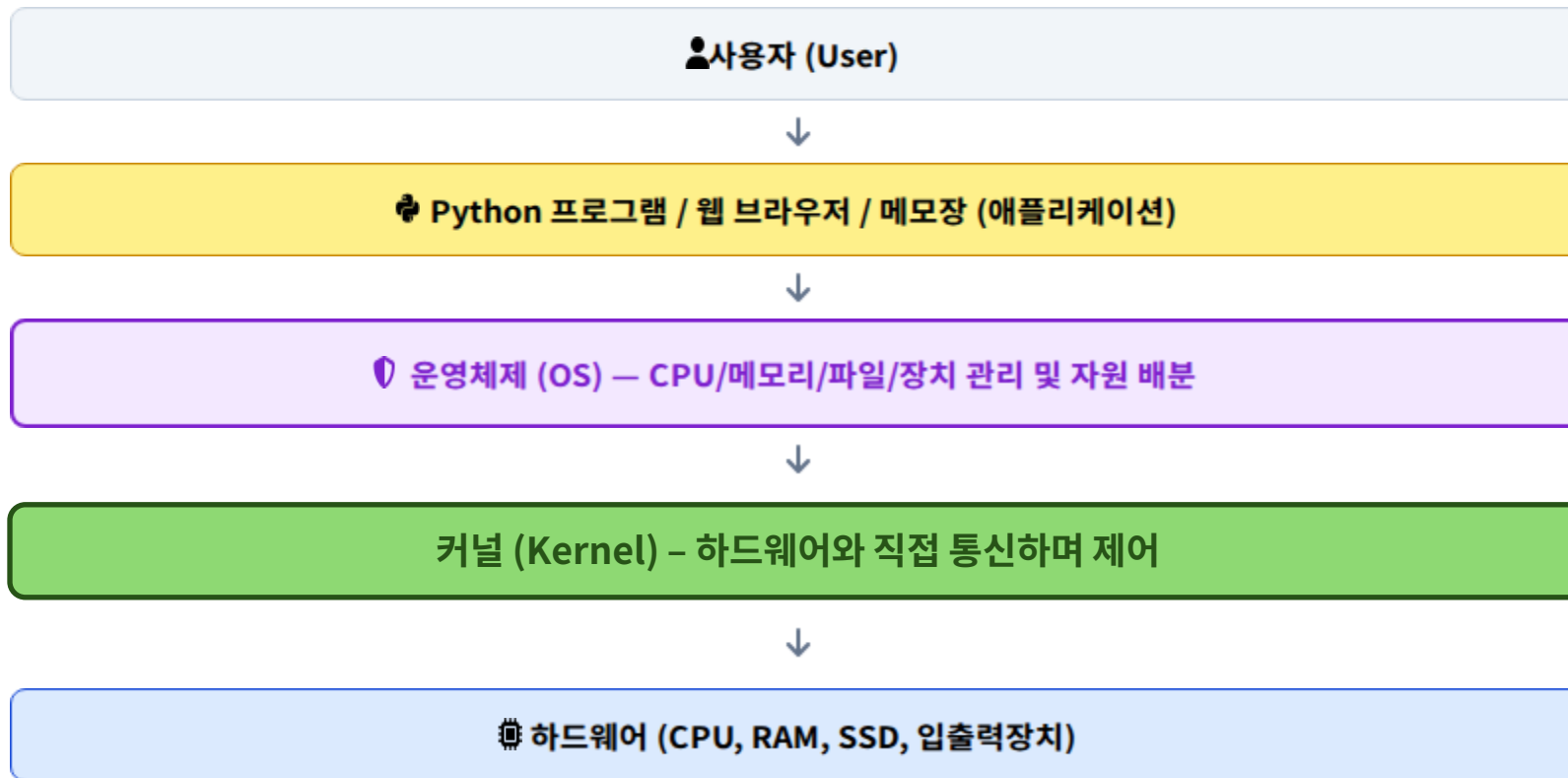
하드웨어를 직접 제어하는 복잡함을 대신 처리해 줌

여러 프로그램이 동시에 안전하게 실행되도록
자원을 배분함

파일을 안전하게 저장하고 불러오도록 관리함

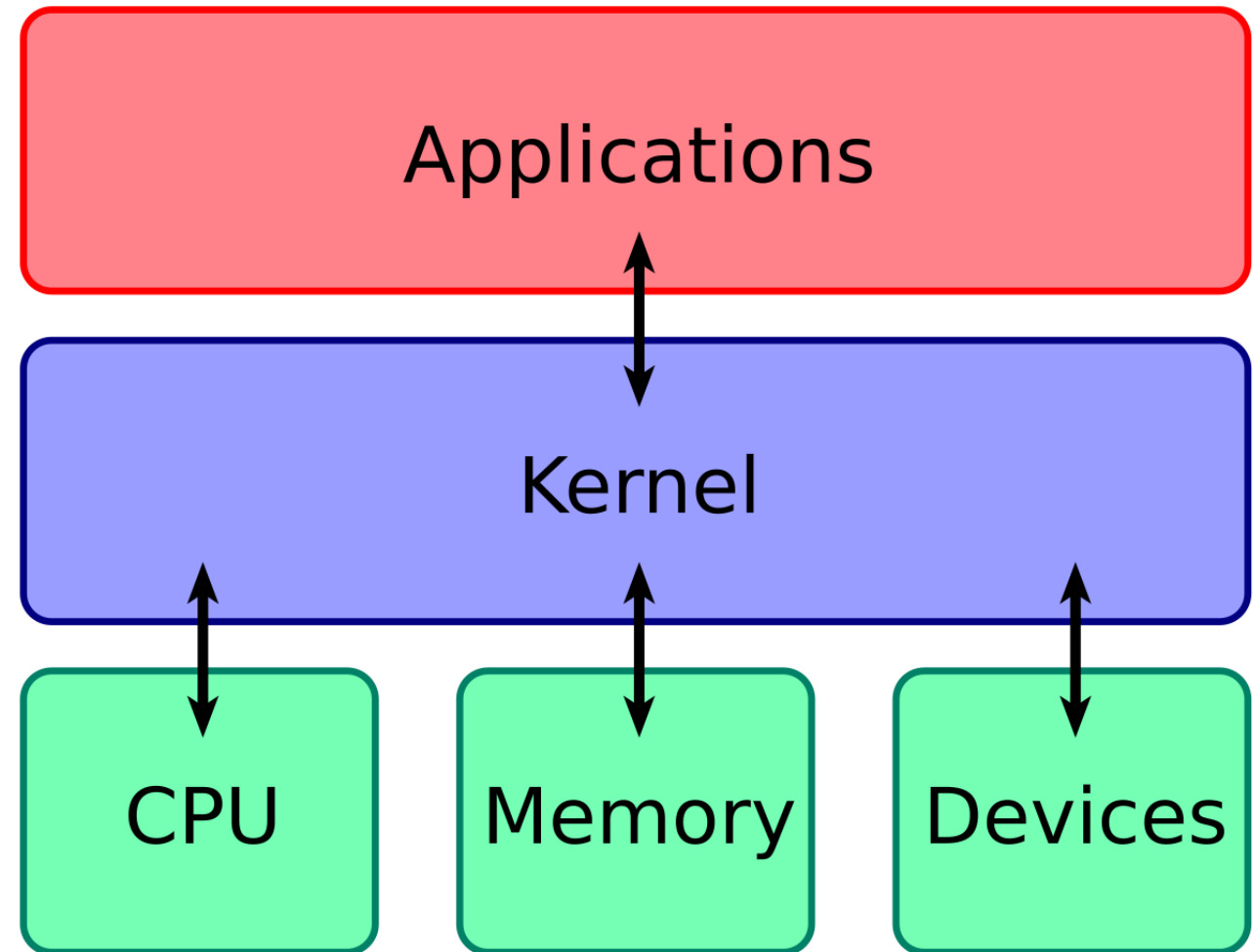


시스템 계층 구조 System Hierarchy



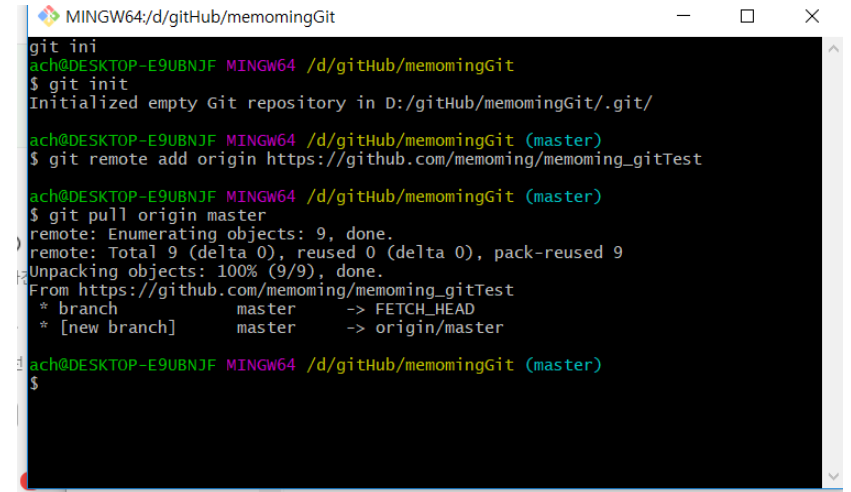
커널 Kernel

- 운영체제의 핵심인 소프트웨어
- 하드웨어 자원을 자원이 필요한 프로그램에 나눠줌
- 작업 제어, 메모리 제어, 시스템 콜 등 수행
- 하드웨어에 직접적으로 명령
- OS마다 커널 구조가 다 다름



셸 Shell

- 커널보다 한 단계 바깥의 소프트웨어
- 이용자의 명령을 해석하고 그 결과를 반환
- 사용자가 OS의 핵심(커널)을 둘러싼 껍질을 통해 커널에 접근한다 해서 Shell 이라 부름
- Git Bash, CMD, PowerShell 등이 대표적인 Shell



```
MINGW64/d/github/memomingGit
git ini
ach@DESKTOP-E9UBNJF MINGW64 /d/github/memomingGit
$ git init
Initialized empty Git repository in D:/github/memomingGit/.git/

ach@DESKTOP-E9UBNJF MINGW64 /d/github/memomingGit (master)
$ git remote add origin https://github.com/memoming/memoming_gitTest

ach@DESKTOP-E9UBNJF MINGW64 /d/github/memomingGit (master)
$ git pull origin master
remote: Enumerating objects: 9, done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 9
Unpacking objects: 100% (9/9), done.
From https://github.com/memoming/memoming_gitTest
* branch      master      -> FETCH_HEAD
* [new branch] master      -> origin/master

ach@DESKTOP-E9UBNJF MINGW64 /d/github/memomingGit (master)
$
```



```
관리자: 명령 프롬프트
Microsoft Windows [Version 10.0.19045.2364]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator>
```

사용자모드 & 커널모드 User Mode & Kernel Mode

상단 영역: User Mode (사용자 모드)

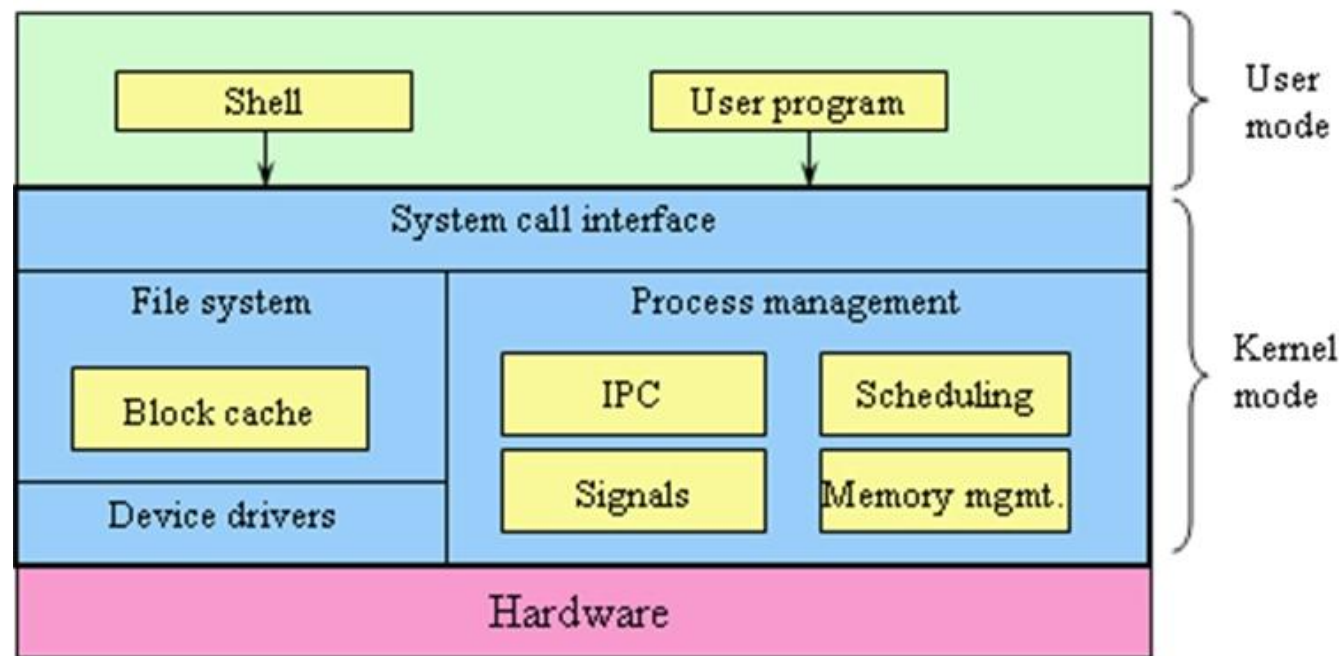
Python 프로그램, 웹 브라우저, 메모장 등 일반 애플리케이션 실행 영역
(하드웨어 직접 접근 불가)

System Call Interface

일반 프로그램이 커널에게 필요한 작업을 요청하는 공식적 통로

하단 영역: Kernel Mode (커널 모드)

운영체제 핵심(커널), 파일 시스템, 메모리 관리, 장치 드라이버
(하드웨어 직접 제어 가능)



일반 프로그램이 위험한 하드웨어 자원을 직접 망가뜨리지 못하도록,
OS가 두 개의 모드로 권한을 엄격히 분리

시스템 콜 System Call

시스템 콜 (System Call)이란?

"일반 프로그램(User Mode)이 운영체제 커널(Kernel Mode)에게 필요한 하드웨어 작업을 요청하는 공식적인 통로"

손님(파이썬 프로그램)이 주방(커널)에 직접 들어가 요리를 할 수 없으므로 웨이터(시스템 콜)를 통해 주문하는 것과 같음

Python 코드



System Call 요청



OS 커널 처리



하드웨어 장치 제어

04

코드 실행

Code Execution

컴파일러 & 인터프리터

Compiler & Interpreter

컴파일러 (Compiler)

프로그램 실행 **전에** 소스 코드 전체를 한 번에 번역합니다.
완벽한 독립 실행 파일(예: .exe)을 생성하는 경우가 많습니다.

대표 언어: C, C++, Go 등

인터프리터 (Interpreter)

코드를 읽으면서 동시에 한 줄씩(또는 중간 형태로) 해석하고 실행합니다.

별도의 실행 파일 빌드 과정 없이 즉시 결과를 확인할 수 있습니다.

대표 언어: **Python**, JavaScript 등



Python이 내부에서 실행되는 과정



CPython 표준 기준

우리가 가장 많이 쓰는 표준 Python(CPython)은 소스코드를 .pyc 형태의 바이트코드로 먼저 바꾼 뒤, 파이썬 가상 머신(PVM)을 통해 CPU가 이해할 수 있는 명령어로 해석해 나갑니다.

CPU가 파이썬 소스 코드를 직접 읽는 것은 절대 아닙니다!

Python이 내부에서 실행되는 과정



```
print("Hello World!")
```



Python Interpreter

2	RESUME	0
3	LOAD_GLOBAL	1 (print + NULL)
	LOAD_CONST	1 ('Hello World!')
	CALL	1
	POP_TOP	
	RETURN_CONST	0 (None)

hello.py
소스 코드



Python 인터프리터
바이트코드 변환

PVM (가상 머신)
중간 형태 실행

Python이 내부에서 실행되는 과정

```
010110
011010
100110
```

Machine Language

```
55
48 89 E5
BF 04 20 40 00
E8 FC FE FF FF
B8 00 00 00 00
5D
C3
```

Hex

```
01010101
01001000 10001001 11100101
10111111 00000100 00100000 01000000 00000000
11101000 11111100 11111110 11111111 11111111
10111000 00000000 00000000 00000000 00000000
01011101
11000011
```

Binary

Python 인터프리터

바이트코드 변환

PVM (가상 머신)

중간 형태 실행



운영체제 & CPU

최종 기계어 처리

Python이 내부에서 실행되는 과정

```
010110
011010
100110
```

Machine Language (기계어)

```
55
48 89 E5
BF 04 20 40 00
E8 FC FE FF FF
B8 00 00 00 00
5D
C3
```

Hex

```
01010101
01001000 10001001 11100101
10111111 00000100 00100000 01000000 00000000
11101000 11111100 11111110 11111111 11111111
10111000 00000000 00000000 00000000 00000000
01011101
11000011
```

Binary



Assembly (어셈블리어)

```
push rbp
mov rbp, rsp
mov edi, 0x402004
call puts
mov eax, 0
pop rbp
ret
```

기계어를 사람이 읽기 쉽게 일대일로
대응하여 표현한 저급 프로그래밍 언어

프로그래밍 언어의 급

Levels of Programming Language

- 프로그래밍 언어는 급이 나뉘어져 있음
- 저급 언어는 저품질이라는 뜻이 아니라, 하드웨어에 가깝다는 뜻
- 저급 언어: 기계어, 어셈블리어
- 중급 언어: C (비공식적인 분류)
- 고급 언어: C++, Python, Java, JS, C#, Go 등

